

I040-2-8461

Open Source Software

02. OSS 개론 및 라이선스

Dr. Kyudong Park

School of Information Convergence



Contents

- [오픈소스의 정의와 역사](#)
- 오픈소스의 철학과 장단점
- 오픈소스 라이선스의 종류

정의

- Open Source Software (OSS)

- 소스코드에 공개적으로 접근할 수 있게 설계되어 누구나 자유롭게 확인, 수정, 배포할 수 있는 소프트웨어
- 소스코드 : 소프트웨어의 설계도

- 비교

- Freeware : 무료로 사용 가능한 소프트웨어, 상업적 이용이 불가능

```
7 string sInput;  
8 int iLength, iN;  
9 double dblTemp;  
10 bool again = true;  
11  
12 while (again) {  
13     iN = -1;  
14     again = false;  
15     getline(cin, sInput);  
16     system("cls");  
17     stringstream(sInput) >> dblTemp;  
18     iLength = sInput.length();  
19     if (iLength < 4) {  
20         again = true;  
21         continue;  
22     } else if (sInput[iLength - 3] != '.') {  
23         again = true;  
24         continue;  
25     } while (++iN < iLength) {  
26         if (isdigit(sInput[iN])) {  
27             continue;  
28         } else if (iN == (iLength - 3)) {  
29             continue;  
30         }  
31     }  
32     if (iN == (iLength - 3)) {  
33         continue;  
34     }  
35 }
```

오픈소스와 개발 방식

- 개발방식 = 분산된 협업
 - Peer review
 - Community-based production
 - 단일 작성자나 기업이 개발하지 않고 커뮤니티가 개발함

History

- 1950 - 1960
 - 컴퓨터 = 하드웨어
 - 소프트웨어는 하드웨어를 사면 딸려오는 것
 - 소프트웨어가 상품으로 취급받던 시기가 아님
 - 하드웨어 아래에서 학계와 기업 연구자들이 공동으로 필요한 프로그램을 제작하고 공유하는 문화
 - 개방/협업적 환경으로 초기 인터넷 기술과 통신 네트워크 프로토콜을 개발



History

- 1960 - 1970
 - 운영체제의 진화
 - 대화형 시스템 (키보드 마우스), 시분할 시스템 => 하드웨어의 효율적 관리
 - 소프트웨어의 제작 비용이 하드웨어 대비 증가
 - 하드웨어 제조사에 독립적인 소프트웨어
 - 소프트웨어에 대한 소유권이 인정되고, 재판매와 재사용이 금지됨 => 하나의 권리로 인정되기 시작
 - IBM 끼워팔기 반독점 소송 (1969)
 - IBM의 대응 : SW를 HW에서 분리하여 판매 및 3% 가격 인하
 - 시스템SW 와 응용SW 개념의 탄생

History

- 1970 - 1980
 - UNIX의 등장
 - 대학에 소스코드 공개
 - 대부분 버전은 유료
 - 영리 소프트웨어 기업의 본격적인 등장
 - 소스코드 = 설계도
 - 소스코드의 철저한 보호
 - 불법복제에 대한 의견

February 3, 1976

An Open Letter to Hobbyists

To me, the most critical thing in the hobby market right now is the lack of good software courses, books and software itself. Without good software and an owner who understands programming, a hobby computer is wasted. Will quality software be written for the hobby market?

Almost a year ago, Paul Allen and myself, expecting the hobby market to expand, hired Monte Davidoff and developed Altair BASIC. Though the initial work took only two months, the three of us have spent most of the last year documenting, improving and adding features to BASIC. Now we have 4K, 8K, EXTENDED, ROM and DISK BASIC. The value of the computer time we have used exceeds \$40,000.

The feedback we have gotten from the hundreds of people who say they are using BASIC has all been positive. Two surprising things are apparent, however. 1) Most of these "users" never bought BASIC (less than 10% of all Altair owners have bought BASIC), and 2) The amount of royalties we have received from sales to hobbyists makes the time spent of Altair BASIC worth less than \$2 an hour.

Why is this? As the majority of hobbyists must be aware, most of you steal your software. Hardware must be paid for, but software is something to share. Who cares if the people who worked on it get paid?

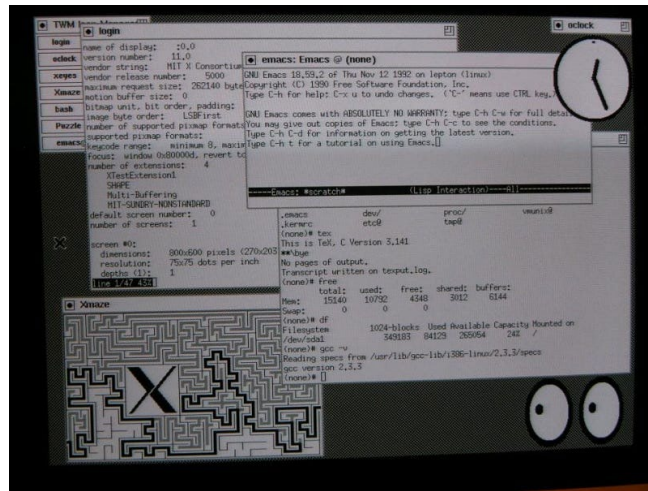
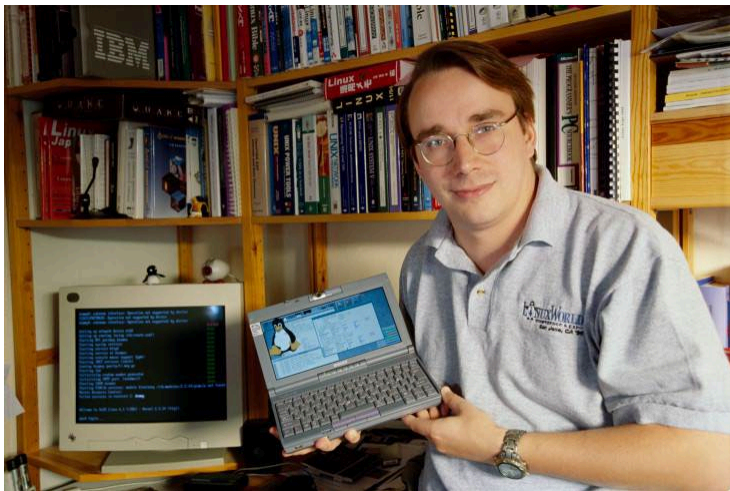
Is this fair? One thing you don't do by stealing software is get back at MITS for some problem you may have had. MITS doesn't make money selling software. The royalty paid to us, the manual, the tape and the overhead make it a break-even operation. One thing you do do is prevent good software from being written. Who can afford to do professional work for nothing? What hobbyist can put 3-man years into programming, finding all bugs, documenting his product and distribute for free? The fact is, no one besides us has invested a lot of money in hobby software. We have written 6800 BASIC, and are writing 8080 APL and 6800 APL, but there is very little incentive to make this software available to hobbyists. Most directly, the thing you do is theft.

History

- 1980 - 1990
 - 자유 소프트웨어 개념의 기원
 - Richard Stallman
 - GNU 프로젝트
 - GNU 선언 : 소프트웨어는 공유되어야 하며 프로그래머는 소프트웨어로 돈을 벌어서는 안 된다.
 - Free Software Foundation 설립
 - free ≠ 무료
 - GPL (General Public License) 정립
- 개작 가능, 원본 배포 의무, 파생물 배포 의무, 파생물 라이선스 의무
- GNU 프로젝트에서 만든 소프트웨어들은 모두 적용됨

History

- 1990 - 2000
 - 인터넷과 Linux 의 보급
 - Linus Torvalds - 운영체제 커널을 만들어서 GPL 라이선스로 배포 (최초의 리눅스)

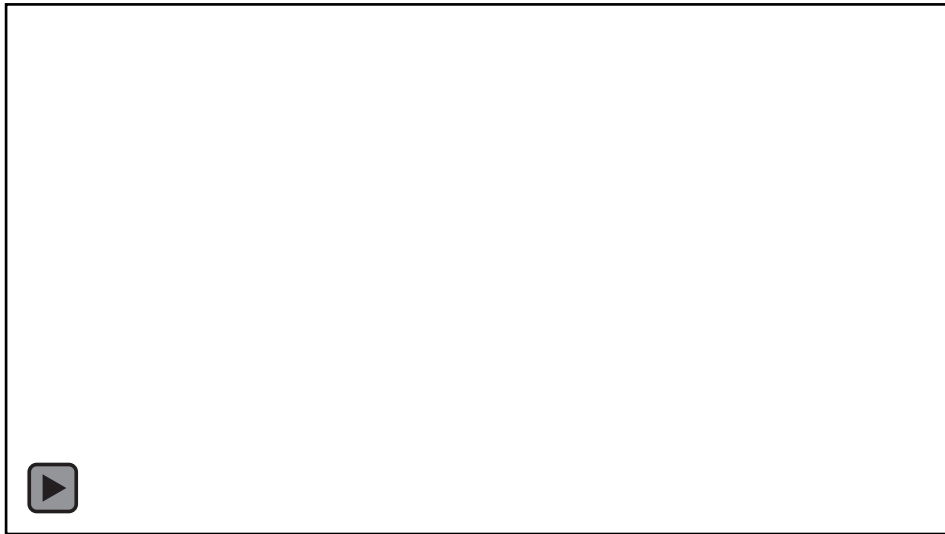


History

- 1990 - 2000
 - 자유소프트웨어 → 오픈소스소프트웨어 (1998)
 - OSI (Open Source Initiative) 결성
 - 오픈소스 소프트웨어의 기준 (Open Source Definition)
 - ① 자유 배포(Free Redistribution)
 - ② 소스코드 공개(Source Code Open)
 - ③ 2차적 저작물 허용(Derived Works)
 - ④ 소스코드 수정 제한(Integrity of The Author's Source Code)
 - ⑤ 개인이나 단체에 대한 차별 금지 (No Discrimination Against Persons or Groups)
 - ⑥ 사용 분야에 대한 제한 금지 (No Discrimination Against Fields of Endeavor)
 - ⑦ 라이선스의 배포 (Distribution of License)
 - ⑧ 라이선스 적용상의 동일성 유지 (License must not be specific to a product)
 - ⑨ 다른 라이선스의 포괄적 수용 (License must not contaminate other software)
 - ⑩ 라이선스의 기술적 중립성 (License must be Technology-Neutral)

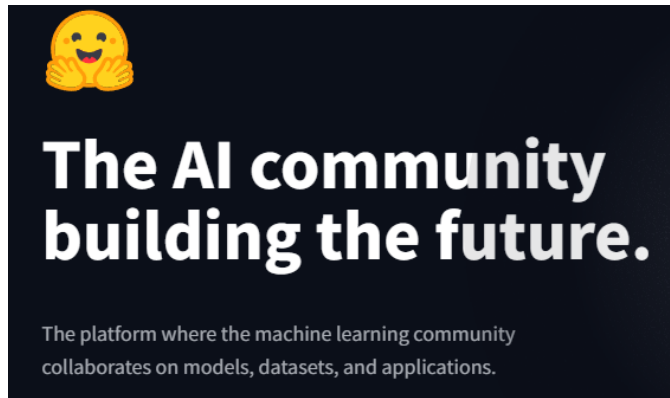
History

- 2000 - 2010
 - 리눅스의 대성공
 - 많은 개발자들이 협력적으로 개발에 참여 = 오류, 취약점의 개선 (약 15,000명의 코드 기여)
 - Redhat, debian, ubuntu
 - 분산버전컨트롤 시스템 (DVCS)
 - Git 의 개발 (Linus Torvalds, 2005)



History

- 2010 - 현재
 - 여러 분야에서 오픈소스 도입
 - DB : MariaDB, 웹서버 : Apache
 - 웹브라우저 : Firefox, 빅데이터관리 : Hadoop
 - 가상화 : docker , CI/CD : Jenkins
 - github 의 인기
 - MS의 github 인수 (2018)
 - 인공지능 역시 오픈소스 문화를 지향하면서 발전
 - huggingface (2016)



오픈소스로 인해 달라지는 개발 방식

- 다른 사람이 개발한 소스코드를 이해하는 능력
- 다른 사람이 개발한 소스코드에서 개선점을 발견하는 능력
- 다른 사람이 개발한 소스코드를 수정하여 가치를 더하는 능력
- 여러 오픈소스들을 연결하는 능력

Contents

- 오픈소스의 정의와 역사
- 오픈소스의 철학과 장단점
- 오픈소스 라이선스의 종류

오픈소스의 철학

- OSI에서 정의한 원칙
- 주요 가치
 - 완전한 개방
 - 자유로운 참여
 - 자발적인 공유



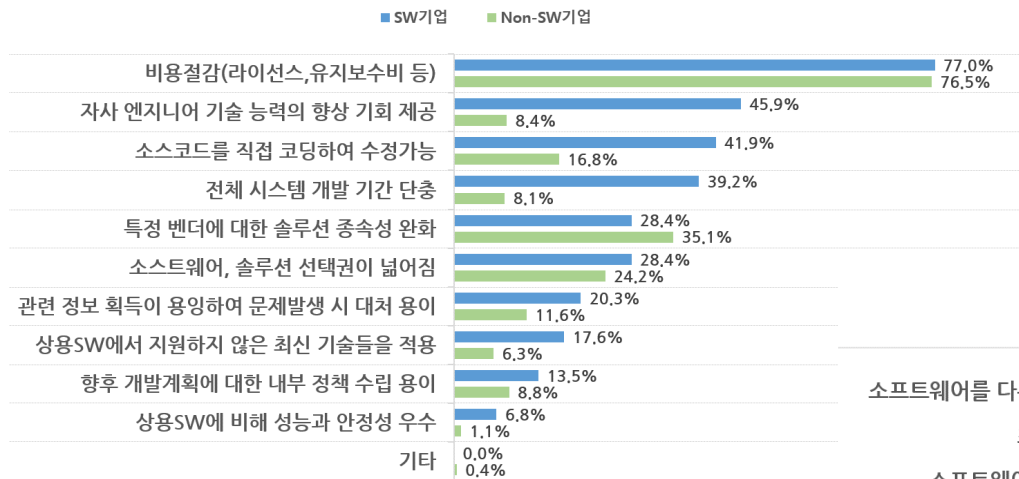
오픈소스 소프트웨어의 장점

- 경제적 효율성
 - 서버도입 및 유지보수 비용 감축만을 고려했을 때 연간 약1,040억원의 경제적 이득
 - 제품개발 시 공개SW를 활용할 경우 시간/인력 등 개발투입비용의 절감 발생
- 종속성 극복 및 시장 경쟁 촉진
 - 어느 한 솔루션의 공급 업체나 기술에 종속될 위험 완화
 - 소비자에게 다양한 대안을 제시
- 기술 혁신
 - 최신 기술 정보 및 문제점과 해결책을 공유하는 형태로 자유롭게 운영되기 때문에 기술발전속도가 빠름
- 인력 양성
 - 소스코드 공개로 인해 선진 기술을 습득하고 개발에 동참하여 개인 역량 향상의 기회 제공

오픈소스 소프트웨어의 장점

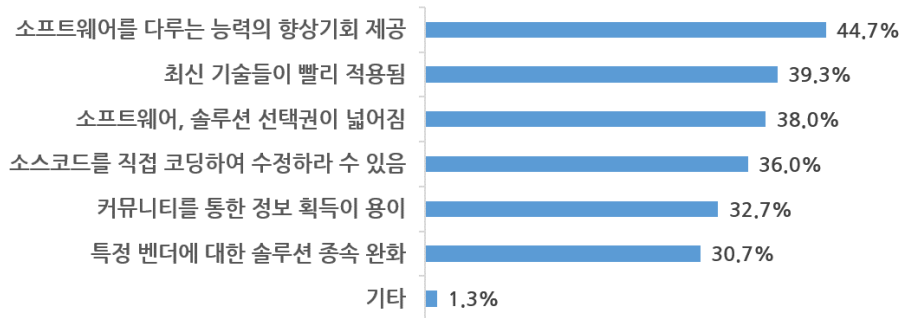
공개SW 활용 장점(기업)

[N=283, 중복응답]



공개SW 활용 장점(개인)

[N=150, 중복응답]



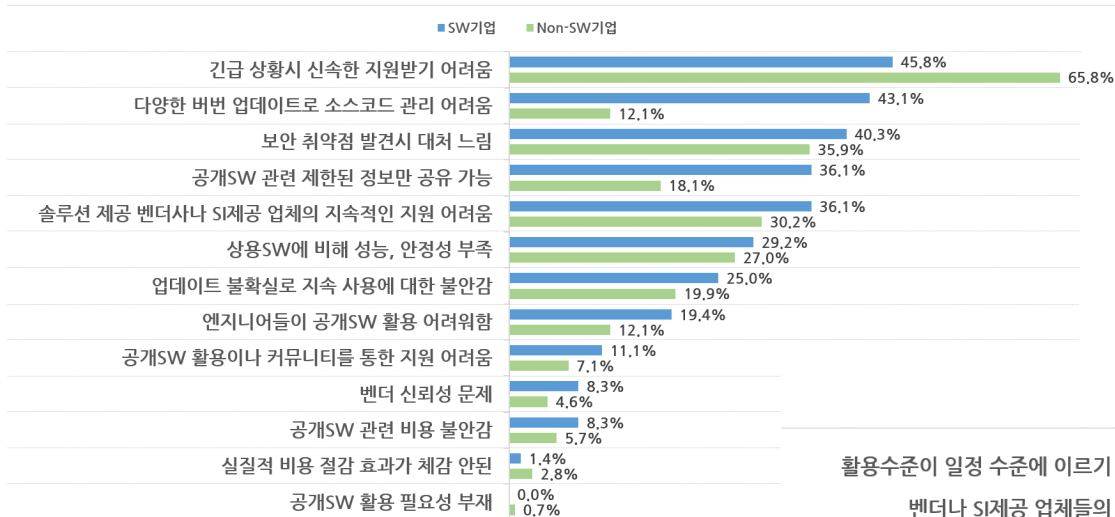
오픈소스 소프트웨어의 단점

- 빈약한 문서
 - 상용 프로그램에 비해 오픈소스는 체계적인 문서를 갖지 않은 경우가 많음
 - 개발과정을 지체시키는 원인이 되기도 함
- 불확실한 로드맵 (개발 청사진)
 - 독점 프로그램에서 볼 수 있는 로드맵을 기대하기 어려움
 - 갑자기 단종되고, 업그레이드가 중단될 수 있음
- 지적재산권
 - 보유하고 있는 특허/핵심 기술을 소스코드에 포함시켜 재배포하려는 경우에도 원 저작자의 라이선스 정책을 지켜야 함

오픈소스 소프트웨어의 단점

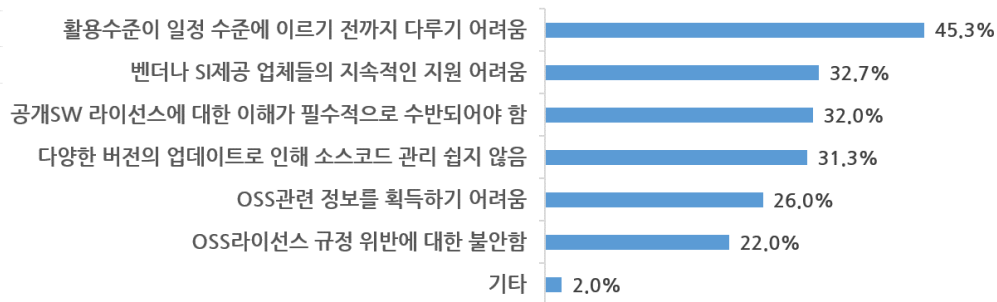
공개SW 활용 단점(기업)

[N=281, 중복응답]



공개SW 활용 단점(개인)

[N=150, 중복응답]



오픈소스 소프트웨어와 경제

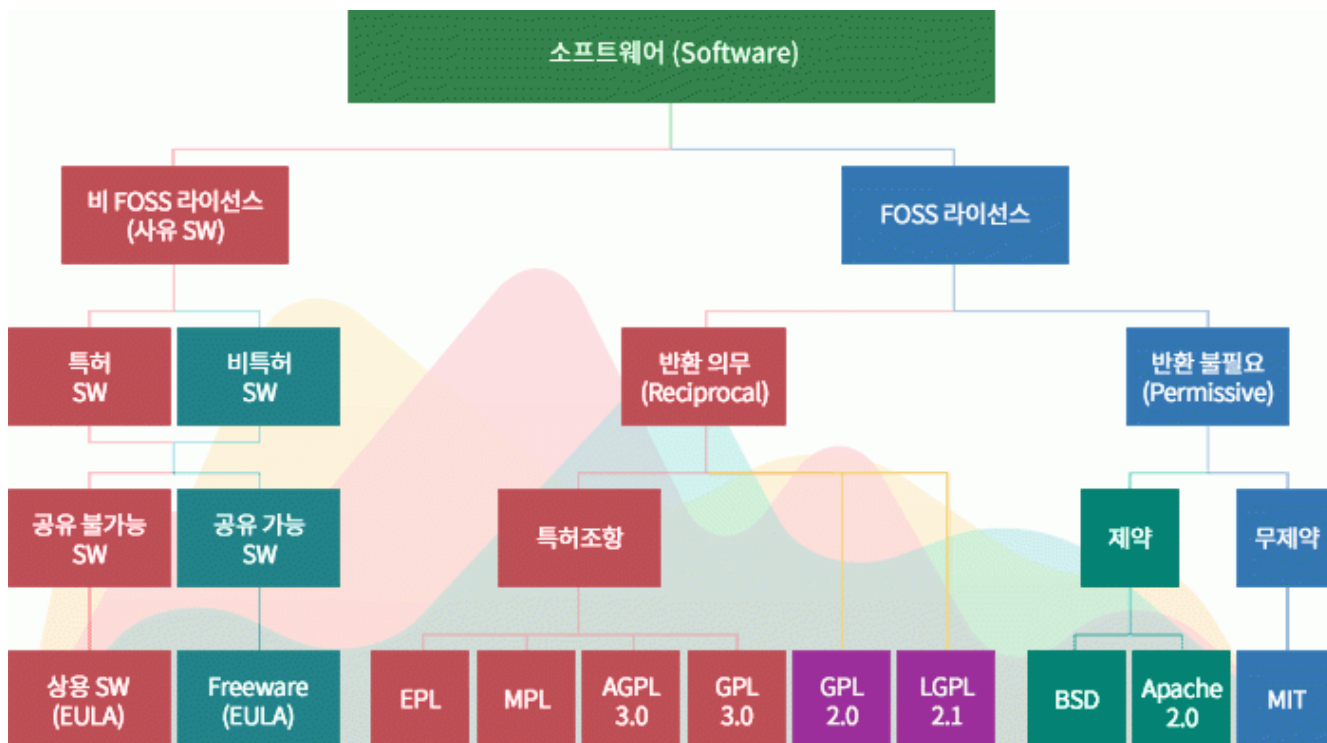
- 개발자 개인 관점에서 미래 수익 창출에 중요한 역할
 - 포트폴리오
 - 최신 기술에 숙달
- 돈을 벌 수 있음
 - 기업용 시장
 - 관련 책, 강좌, 전문가 인증 제도
 - e.g.) Apache 재단이 기업에게 Hadoop 을 교육하고 돈을 받음 (소프트웨어 사용 명목은 아님)
 - 기술 지원
 - 업데이트 주기가 빠를 때, 호환 및 적응 시간 등에 대한 컨설팅

Contents

- 오픈소스의 정의와 역사
- 오픈소스의 철학과 장단점
- 오픈소스 라이선스의 종류

OSS 라이선스의 종류

- 소스가 공개되어 사용, 복제, 배포, 수정 할 수 있지만, 원 저작자의 라이선스 규칙에 따라야 함



OSS 라이선스에 공통 적용 사항

- 라이선스 및 저작권 명시
 - 배포할 때 라이선스 및 저작권을 명시해야한다.
- 보증 책임 없음
 - 특정한 목적에 대한 프로그램의 적합성이나 상품성 여부에 대한 보증을 포함한 어떠한 형태의 보증도 명시적이나 묵시적으로 설정되지 않은 “있는 그대로의” 상태로 이 프로그램을 배포한다. 프로그램의 품질과 성능에 관련된 모든 위험은 당신에게 있다. 프로그램에 결함이 있는 것으로 밝혀지면, 이에 따라 필요한 보수 및 복구를 위한 제반 경비는 당신이 부담해야 한다.

OSS 라이선스의 종류

- GPL (General Public License) 의 기초
 - 소프트웨어를 어떠한 목적에든 사용할 수 있는 자유
 - 소프트웨어를 사용자의 친구 또는 이웃과 공유할 수 있는 자유
 - 소프트웨어를 사용자의 필요에 따라 수정할 수 있는 자유
 - 수정한 소프트웨어를 공유할 수 있는 자유
- GPL 2.0 (1991년 6월 발표)
 - 가장 많은 OSS 가 채택하고 있음
 - 소스코드 공개 의무가 발생하려면 물리적인 프로그램의 이동인 '배포'가 발생해야 함
 - 프로그램을 서버에 저장하고, API 형식으로만 서비스한다면 공개 의무 없음
 - 의무 사항들도 타 라이선스에 비해 엄격
 - 배포 시 저작권 표시, 보증 책임 없음 표시, GPL 배포라는 사항 명시
 - 수정 시 재공개, 2차 라이선스 금지

OSS 라이선스의 종류

- GPL 3.0 (2007년 6월)
 - GPL v2 공표 이후 17년 동안 기술 발전과 소프트웨어 상황을 반영함
 - Tivoization
 - 미국의 Tivo라는 회사는 케이블방송 또는 위성방송 등의 방송프로그램을 하드디스크에 녹화하는 제품을 출시함
 - 해당 제품은 리눅스 기반으로 개발되었지만(GPL 2.0), 이용자들이 소스코드를 수정하여 넣으면 기기가 꺼져버림
 - GPL 3.0에서는 소프트웨어를 수정하여 설치하고 실행하는데 필요한 방법을 모두 공개하여야 함



OSS 라이선스의 종류

- GPL 3.0 (2007년 6월)
 - DRM (Digital Rights Management) 우회 도구 개발
 - FSF는 GPL 프로그램이 DRM으로 사용되는 것에 반감을 가짐
 - GPL 라이선스가 있는 프로그램에 DRM 이 있을 때에는 이를 우회하는 도구 역시 만들어도 됨



DRM을 다운로드하면 어떻게 되는거죠?

한국씩 구매하는 MP3와 달리 DRM을 다운로드하면
다운받은 플랫폼을 통해 데이터 없이
음원을 들을 수 있습니다.

다만, 저작권이 걸려있기 때문에 평생 소장이 불가하며
이용권 사용기간 내에만 사용할 수 있습니다.

OSS 라이선스의 종류

- GPL 3.0 (2007년 6월)

- 특허로부터의 강력한 보호

- GPLv3를 적용한 소프트웨어를 제작 또는 수정한 양도인은 모든 양수인들에게 GPL이 제공하는 권리를 실행할 수 있도록 모든 특허 라이선스를 제공하여야 한다.

- 회사 A가 자체 특허를 구현한 a와 GPL2.0 기반 오픈소스b 를 결합하여 올려놓음. 개발자 B가 이 코드를 사용한다면?
 - GPL3.0 오픈소스b 라면, 그 회사 A는 특허 a의 권리를 주장할 수 없음을 명시함.
 - 그럼에도 불구하고 회사 A가 사용한 개발자B를 고소한다면?
 - 회사 A의 GPL 라이선스가 종료됨 = 무단으로 b를 활용한 것으로 간주

- 방어적 특허의 개념



OSS 라이선스의 종류

- AGPL (Affero GPL)

- 네트워크상 소프트웨어에 대해서도 소스 코드 공개



- LGPL (Lesser GPL)

- 소스코드를 공개해야한다는 부담을 줄이기 위해 라이브러리와 모듈로의 링크를 허용
- 응용프로그램에 라이브러리를 Linking 시킬 경우, 응용프로그램의 소스를 공개할 필요 없음
 - LGPL 라이브러리의 일부를 수정하는 경우, 수정한 라이브러리의 소스코드 공개
- 배포 시 저작권 표시, 보증 책임 없음 표시



OSS 라이선스의 종류

- BSD (Berkeley Software Distribution)
 - 허용 범위가 넓고, 소스 코드를 공개하지 않아도 됨
 - 미국 정부 재원으로 지원 받은 연구 결과물을 다시 대중에게 환원한다는 의도
 - 배포 시 저작권 표시, 보증 책임 없음 표시, 상업적 이용 가능
- Apache 라이선스
 - ASF (Apache Software Foundation) 에서 제공하는 라이선스, BSD 와 유사
 - 아파치 재단의 모든 오픈소스 소프트웨어 이용 시 적용
 - Apache 이름에 대한 **상표권**을 침해할 수 없음
 - 특허 조항 관련 GPL 3.0과 호환 가능
 - 배포 시 저작권 표시, 보증 책임 없음 표시, 상업적 활용 가능

BSD



OSS 라이선스의 종류

- MPL 라이선스

- Mozilla Public License
- 소스코드와 실행파일의 저작권 분리
- 수정한 코드는 MPL로 공개하고 원저작자에게 수정 부분에 대해 알려야 함
- 실행 파일 자체는 독점 라이선스로 배포 가능



- MIT 라이선스

- MIT 에서 제정, 학생들을 돕기 위해 만든 라이선스
- 개작된 소프트웨어를 반드시 오픈소스로 해야할 필요도 없음
- 완전히 열려 있는 라이선스



OSS 라이선스의 종류

- Beerware License



BEERWARE

OSS 라이선스의 종류

- 주요 라이선스 비교

- AGPL
- GPL
- LGPL
- MPL
- Apache
- BSD
- MIT

구분	무료 이용가능	배포 허용가능	소스코드 취득가능	소스코드 수정가능	2차적 저작물 재공개 의무	독점SW와 결합가능
GPL	●	●	●	●	●	X
LGPL	●	●	●	●	●	●
MPL	●	●	●	●	●	●
BSD license	●	●	●	●	X	●
Apache llicense	●	●	●	●	X	●

Thank you

Q & A